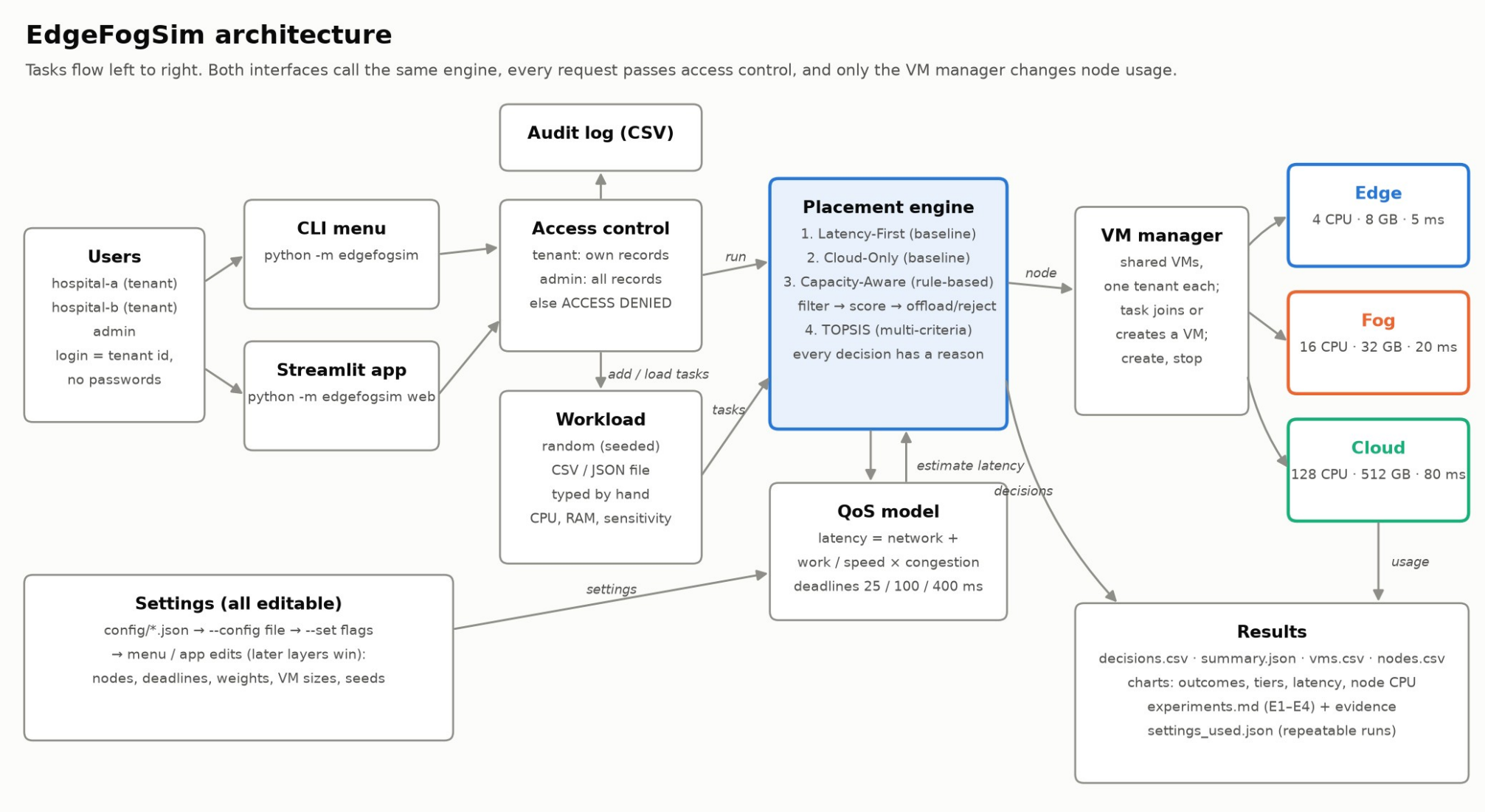
EdgeFogSim

Where should a task run: Edge, Fog or Cloud? A rule-based placement simulator with VMs, tenant security and QoS results.

ARM 305 – Cloud, Dev, Edge and Fog Computing · Mini Project · Live: cdef-project.streamlit.app

Team member	Enrollment number	Contribution
Moksh	06719051624	Placement policies, simulator, experiments
Bhavya Bisht	05019051624	VM manager and security, Streamlit app
Dhruv Sharma	05719051624	Results export and charts, testing
Bhuwan Chugh	04219051624	Settings system, workload generator and task files, command-line menu; report, slides, architecture diagram

Every task passes access control, a policy and a shared VM



How latency and the placement score are calculated

Latency of a task on a node

$$L = d + (W \div s) \times C \qquad C = 1 \div (1 - u)$$

L	latency: time until the answer comes back (ms)
d	network delay to the node: edge 5, fog 20, cloud 80 ms
W	work units: how much processing the task needs
s	node speed: edge 0.6, fog 1, cloud 2 (W ÷ s = ms of work)
C	congestion: 1 when idle, 2 at half load, 10 at 90% load
u	node CPU load after adding the task, 0 to 1 (capped at 95%)

Placement score (Capacity-Aware)

$$S = 0.5 \cdot T + 0.3 \cdot (D - L) \div D + 0.2 \cdot (1 - u)$$

T	tier preference: 1st choice 1, 2nd 0.5, 3rd 0
D	deadline by sensitivity: high 25, medium 100, low 400 ms
$(D-L)\div D$	share of the deadline left over; success means L ≤ D
$1 - u$	share of the node's CPU still free after adding the task

Worked example: T01 on edge-1

2 CPU · W = 4 · high → D = 25 ms

u = (0 + 2) ÷ 4 = 0.5
C = 1 ÷ (1 − 0.5) = 2
L = 5 + (4 ÷ 0.6) × 2
  = 5 + 6.67 × 2 = 18.3 ms
18.3 ≤ 25 → on time
S = 0.5·1 + 0.3·0.27 + 0.2·0.5
  = 0.68

Other nodes: fog-1 0.43. Highest score wins, so T01 goes to edge-1.

Capacity-Aware: filter, score, then offload or reject

1

Prefer tiers by urgency

high → Edge, Fog, Cloud · medium → Fog first · low → Cloud first

2

Filter out nodes

no room for the task's VM, CPU load over 85%, or L > D (the deadline would be missed)

3

Score the rest

S = 0.5·T + 0.3·(D − L) ÷ D + 0.2·(1 − u); highest wins

4

Offload or reject

not first-choice tier = offload, with the reason; nothing passes (even at 100% load) = reject, naming the failed check on each node

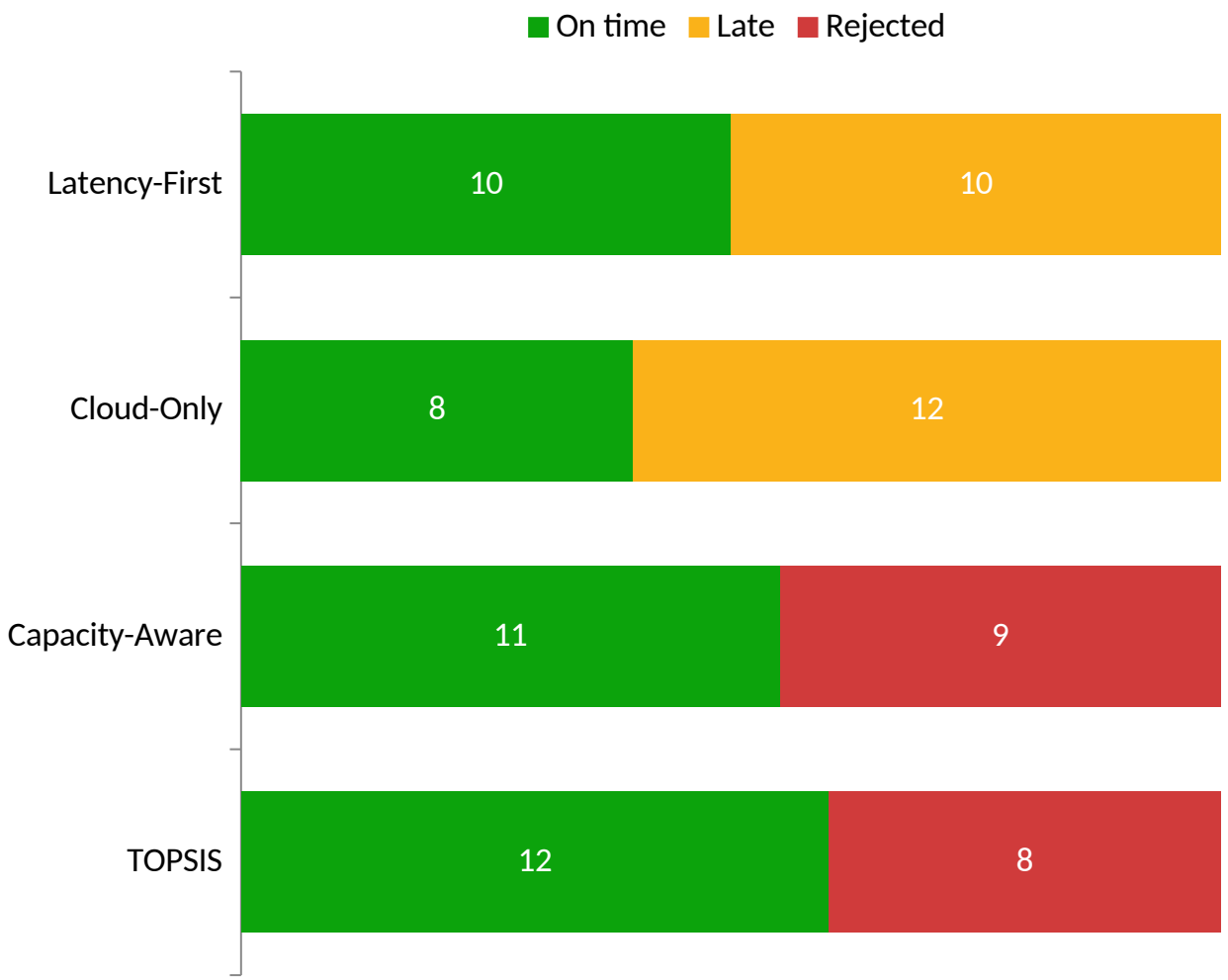
Sample tasks on empty nodes

Task	Needs	Result
T01 (high)	2 CPU, W 4	edge-1, 18 ms
T02 (high)	2 CPU, W 4	fog-1 (offload), 25 ms
T03 (low)	8 CPU, W 40	cloud-1, 101 ms
T04 (high)	20 CPU, W 4	rejected

Compared against

Latency-First: nearest node with room, ignores deadlines
Cloud-Only: every task to the cloud
TOPSIS: ranks nodes on latency, free CPU and free memory together

Smart policies run 0 late; baselines run 10 and 12 of 20 late



0

late with Capacity-Aware

10

late with Latency-First

20 generated tasks, seed 42 (11 high, 5 medium, 4 low sensitivity).

Overload, security and node changes behave as expected

E2 · Overload

Edge already 90% busy, then 10 urgent tasks

4	6	0
offloaded	rejected	late

Offloaded tasks went to Fog in time; every rejection names the check that failed.
Latency-First on the same burst: 1 on time, 9 late.
No node is ever allocated beyond its capacity.

E3 · Unauthorised access

hospital-b tries another tenant's task and VM

DENIED read task T01

DENIED read VM vm-0001

DENIED stop VM vm-0001

Each refusal is one line in the CSV audit log. The VM keeps running; the admin can still read both records.

E4 · Node changes

Same 20 tasks, one node changed at a time

edge-1 cpu_cores 4 → 8

on time 11 → 12
rejected 9 → 8
tasks on edge 1 → 2

cloud-1 base_latency_ms 80 → 150

on time 11 → 9
rejected 9 → 11
avg latency 70.9 → 96.6 ms

What we learned

- All 4 experiments matched their expected outcome (19/19 checks)
- Checking the deadline before placing beats placing on the nearest node
- Shared tenant VMs and one access rule keep tenants apart
- Every number is a setting; runs are repeatable from settings_used.json

Limitations

- No task duration or arrival time; tasks run until their VM stops
- Latency is a formula, not a measurement
- Greedy, one task at a time; GA/PSO or Q-learning could do better
- No VM migration; login is a tenant id by design

Try it: python -m edgefogsim · python -m edgefogsim web · python run_experiments.py